

Reasoning Primitives in Hybrid and Non-Hybrid LLMs: Do Architectural Differences Yield Advantages in State-Tracking and Recall?

Anonymous ACL submission

Abstract

Reasoning in large language models is often discussed as a single capability, but some of its gains may stem from simpler underlying operations. We examine two such primitives, recall and state-tracking, through five controlled task families centered on state-based recall, and compare matched transformer and hybrid architectures with and without reasoning augmentation. Across the suite, reasoning-augmented variants substantially outperform instruction-only variants, often by large margins. This pattern is consistent with the *State over Tokens* view: externalized reasoning traces help because they carry the intermediate state forward in token space. By contrast, hybrid inductive bias does not yield a uniform advantage in accuracy once reasoning tokens are available. When architectural differences do appear, they follow task structure: the hybrid Think model is more robust on strictly sequential chained updates, whereas the transformer Think model is more robust on flat multi-hop retrieval. We therefore cast the main contribution of this study as a descriptive account of what drives performance on state-based recall tasks: reasoning-token augmentation appears to be the dominant factor, while hybrid advantages are narrower, task-dependent, and potentially more about inference efficiency than overall capability. We also release the codebase and data required to reproduce these results (Appendix A).

1 Introduction

Early work on Chain-of-Thought reasoning established that inducing large language models (LLMs) to generate intermediate steps often improves performance on complex tasks (Wei et al., 2022). Subsequent work expanded this picture across prompting strategies (Yao et al., 2023), architectural settings (Geiping et al., 2025), and training regimes (Wang et al., 2025; Yang et al., 2025; Bakouch et al., 2025; Zeng et al., 2026). The harder question

is why reasoning tokens help. The dominant narrative is that they let the model faithfully explain its own reasoning process, but that account is difficult to test and does not explain why reasoning traces can look coherent without transparently mirroring the underlying computation.

A broad body of literature treats reasoning in LLMs as structured inference via intermediate textual computation (Liu et al., 2025), but the mechanism underlying the gains remains contested (Kambhampati et al., 2025). Recent work (Levy et al., 2025) offers a useful lens: the **State over Tokens** (SoT) formulation, which argues that reasoning tokens help because they let models carry intermediate state forward in token space. Across autoregressive generation, internal activations do not persist, but the token prefix does. Intermediate tokens, therefore, become the durable carrier of state across successive forward passes. On this account, reasoning helps not because the model faithfully explains itself, but because extra tokens allow it to externalize partial results, condition on them again, and turn a sequence of bounded computations into a cumulative one. This naturally raises an architectural question: if reasoning gains come from storing state in token space, when should built-in mechanisms for persistent state still matter?

Once the problem is framed in these terms, the architectural question becomes more concrete. Standard softmax-based transformers excel at content-based retrieval but face well-known scaling constraints: full self-attention requires $O(n^2)$ computation and memory, and even with KV caching, sequence length remains a bottleneck as memory grows with stored keys and values (Fichtl et al., 2025; Li et al., 2024). Recurrent and state-space architectures offer more favorable asymptotic profiles, but at the cost of precise content-addressable retrieval (Yang et al., 2024b,a; Dao and Gu, 2024). This trade-off motivates *hybrid architectures*, which combine both mechanisms (Merrill

et al., 2026). In that account, attention supports fine-grained recall over static context, whereas recurrent components support a persistent, efficiently updated latent state. The two are best understood as complementary inductive biases rather than as a single performance hierarchy.

These observations suggest that if reasoning tokens improve performance by externalizing state, then the relevant underlying capabilities may not be “reasoning” in the abstract but simpler operations that support stateful computation. We therefore study two such primitives, **Recall** and **State-tracking** (Appendix B), and ask: once reasoning is decomposed into these computational demands, does performance primarily depend on reasoning augmentation itself, or do architectural differences become visible in systematic ways?

We evaluate matched transformer and hybrid models, with and without reasoning augmentation, on five procedurally generated tasks that probe the same underlying challenge: state-based recall under different surface forms and computational demands. The tasks are not five unrelated benchmarks, but controlled variations on the same problem. Each varies along two axes of difficulty: m (retrieval complexity) and n (state maintenance complexity). Across the suite, reasoning augmentation drives the largest gains, while architectural differences are conditional and become visible only in particular task regimes. When hybrids do have an advantage, it may primarily concern inference efficiency rather than overall accuracy. The contribution of the paper is therefore a descriptive account of when reasoning tokens suffice and when task structure makes architectural differences visible on state-based recall problems.

2 Methods

2.1 Recall and State-Tracking as Reasoning Primitives

To make this notion of reasoning operational, we decompose it into two complementary primitives following (Merrill et al., 2026): **State-tracking**, the maintenance and update of structured variables across sequential transformations, and **Recall**, the high-precision retrieval of specific information from context (formal definitions in Appendix B). Their combination yields **State-based recall**, where the retrieval address is itself produced by a sequence of state updates. In that setting, the task is not reducible to either primitive alone: pure

attention lacks stable iterative state evolution, while pure recurrence compresses away fine-grained addressability. This framing lets us describe the task suite in terms of computational demands rather than benchmark labels. Appendix D provides full task descriptions and examples.

2.2 Experimental Design

We evaluate matched transformer and hybrid models, with and without reasoning augmentation, on five procedurally generated tasks that probe the same underlying challenge: state-based recall under different surface forms and computational demands. The tasks are controlled variations of a single problem, each varying along two difficulty axes: m (retrieval complexity) and n (state-maintenance complexity).

Models. We compare OLMo3-7B (Olmo et al., 2025) and OLMo-Hybrid-7B (Merrill et al., 2026), a matched pair sharing the same data mixture, optimization schedule, and training recipe. For each architecture, we evaluate an instruction-tuned and a reasoning-augmented variant: *OLMo-3-7B-Instruct/Think* and *OLMo-Hybrid-Instruct/Think-SFT-7B*. This design lets us separate the effect of reasoning augmentation from the effect of architectural class. Architecture and inference details are deferred to Appendix E.

Evaluation. All tasks are free-form generation scored by exact matching. Difficulty is parameterized by m (retrieval complexity) and n (state maintenance complexity), with 1000 instances per (m, n) bin (500 for Dyck). All tasks require JSON output `{"answer": "A|B|C|D"}`. We report *accuracy* and *parse rate* (the fraction of parseable responses) separately so that generation failures are not conflated with genuine errors.

Tasks. We evaluate five task families ordered from the minimal baseline to increasingly demanding variants of the same broader problem (Table 1; details in Appendix D). **OLMo Original** (Merrill et al., 2026) is the replication baseline. **Astro Recall** preserves the same pointer-tracking structure with table lookup. **Dyck** isolates state-tracking. **Collisions** introduces chained sequential updates, and **DAG Arithmetic** stresses flat multi-hop retrieval over a growing registry (Motwani et al., 2026). Together, the suite shows how performance changes as the balance between retrieval and state maintenance shifts.

Task	Description
OLMo Original	Bit value after pointer swaps in a synthetic array
Astro Recall	Planet name after variable swaps on orbital periods
Dyck Language	Correct closing bracket in a masked sequence
Collisions	Final velocity after an elastic collision chain
DAG Arithmetic	Variable value after layered arithmetic dependencies

Table 1: Task descriptions and difficulty parameters. For each task, m controls retrieval complexity and n controls state-maintenance complexity: OLMo Original (m = array length, n = # swaps), Astro Recall (m = # table rows, n = # swaps), Dyck Language (m = stack depth, n = sequence length), Collisions (m = # particles, n = # collisions), and DAG Arithmetic (m = # variables, n = # layers).

3 Results

The results are most informative when read comparatively across tasks rather than as isolated leaderboards. Two patterns organize the section: reasoning augmentation is the main source of gains, and architectural differences, when they appear, track task structure rather than difficulty alone. We therefore present the tasks from the OLMo baseline to settings that place increasingly distinct demands on state maintenance and retrieval.

OLMo Original. The replication baseline already establishes the main pattern. The transformer Instruct model consistently exceeds the hybrid Instruct model (0.43–0.53 vs. 0.32–0.43), but once reasoning augmentation is introduced, the two Think variants start near ceiling at (4, 4) and decline in parallel toward chance by (64, 64), without a consistent architectural difference in accuracy. Transformer variants also maintain higher parse rates, though this does not translate into an accuracy advantage at the Think level (Appendix C, Figures 9, 10).

Astro Recall. A more naturalistic retrieval surface leaves the overall story intact. Both Instruct variants remain near chance across the full range while maintaining high parse rates. Both Think variants perform strongly at low difficulty (≈ 0.95 – 0.97 at (4, 4)) but, by (32, 32), increasingly exhaust the inference budget before producing a parseable answer, with the hybrid Think model reaching near-zero parse rate earlier (Appendix C, Figures 5, 6). This reinforces the importance of rea-

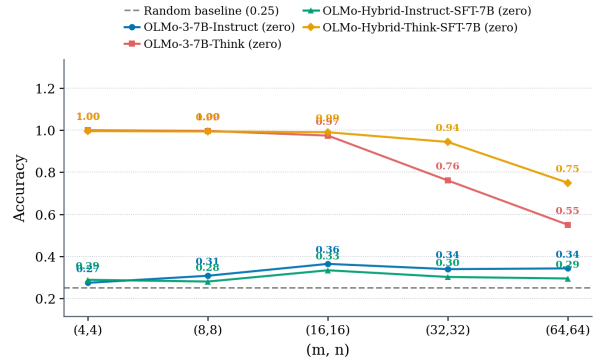


Figure 1: Collision Simulator accuracy.

soning augmentation while also showing that reasoning tokens alone do not eliminate failure once both retrieval and state-maintenance demands become large.

Dyck Language. Dyck isolates the state-tracking side of the decomposition and provides the clearest case in which architecture may matter. Both Instruct variants stay close to the 0.33 random baseline. Among Think variants, OLMo-Hybrid-Think-SFT-7B attains the strongest results in the paper (peak 0.79 at (2, 128) and never below 0.38), whereas OLMo-3-7B-Think is more variable (peak 0.67 and 0.22 at (4, 2048)); the hybrid Think model’s parse rate declines at high difficulty even when accuracy remains comparatively strong (Appendix C, Figures 7, 8). The pattern is consistent with a possible hybrid advantage in pure state-tracking, but we treat it as suggestive rather than general.

Collision Simulator. Collision Simulator adds back state-based recall with strictly chained sequential updates, making it the strongest test of whether architectural differences follow task structure. Both Instruct variants remain near chance (0.27–0.40). At low difficulty both Think variants are near ceiling, but beyond (16, 16) they separate: OLMo-3-7B-Think reaches 0.49 at (32, 32) and increasingly exhausts the inference budget before producing a parseable final answer at (64, 64), whereas OLMo-Hybrid-Think-SFT-7B retains 0.94 and 0.75 at the same settings (Figures 1, 2). This is the strongest evidence in the main results that hybrids can help when success depends on sustaining a long chain of sequential state updates.

DAG Arithmetic. DAG Arithmetic stresses a different part of the state-based recall space:

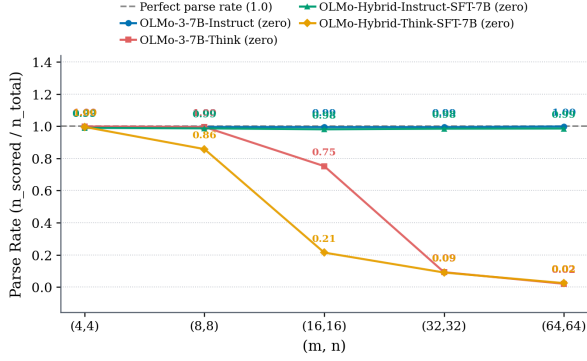


Figure 2: Collision Simulator parse rate.

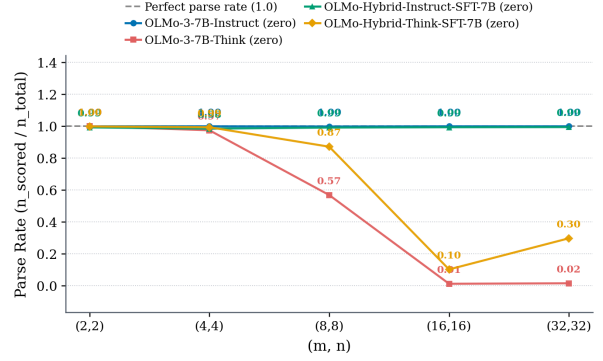


Figure 4: DAG Arithmetic parse rate.

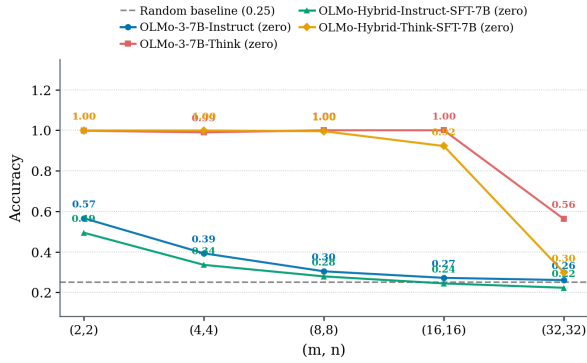


Figure 3: DAG Arithmetic accuracy.

flat multi-hop retrieval over an expanding registry rather than long sequential overwriting. The Instruct variants begin above chance at (2, 2) and approach the 0.25 baseline by (32, 32) with little separation. The Think variants are both perfect from (2, 2) through (8, 8), then diverge in the opposite direction to collisions: at (32, 32) OLMo-3-7B-Think retains 0.56 while OLMo-Hybrid-Think-SFT-7B reaches 0.30, even though the hybrid Think model maintains the higher parse rate throughout (Figures 3, 4). This reversal argues against any single architectural ranking and instead supports a task-dependent interpretation, with transformers appearing more robust when the challenge is content-addressable multi-hop retrieval.

4 Discussion & Conclusion

Across the five tasks, two claims are most directly supported by the data. First, reasoning augmentation is the dominant driver of performance gains: Think variants consistently outperform Instruct variants, often by large margins. This pattern is consistent with the SoT interpretation, in which externalized traces help by carrying the intermediate state in token space. Because the task suite was de-

signed to stress recall and state-tracking, the results also suggest that these primitives capture part of what current “reasoning” improvements rely on.

Second, the advantages of hybrids are conditional rather than general. On the OLMo Original baseline, the two Think models are effectively matched in accuracy, and across the suite, no uniform hybrid advantage emerges. Dyck suggests a possible hybrid benefit in pure state-tracking, while Collisions and DAG Arithmetic reveal opposite advantages depending on whether the task emphasizes sequential state propagation or flat multi-hop retrieval. This pattern is more consistent with task-dependent inductive biases than with a single architectural hierarchy.

The broader implication is that hybrids may matter most for efficiency rather than for uniformly higher capability. Long-context reasoning places strain on KV-cache growth, memory bandwidth, and throughput, all of which scale poorly under full quadratic attention. By mixing sparse full-attention layers with recurrent-style state compression, hybrids can reduce prefilling and decoding costs while preserving enough global interaction to remain competitive on complex tasks. Our contribution is therefore both empirical and methodological: we provide a controlled comparison of transformer and hybrid reasoning models, and we introduce a compact suite that tests architectural claims using recall and state tracking rather than broad benchmark aggregates. Taken together, the results suggest that the main benefits of hybrid models may lie less in qualitatively different reasoning abilities than in more efficient support for those abilities at scale.

5 Limitations

This study has several limitations that the authors acknowledge. First, the architectural comparison is restricted to a single matched 7B model pair, which makes the comparison relatively controlled and fair, but also limits generalization: the observed patterns may be specific to the OLMo3 family, this particular scale, or the associated training recipe and hybrid design choices.¹ Second, all evaluations are conducted on synthetic tasks. This choice is intentional, since the benchmark is designed to isolate recall, state-tracking, and their composition into state-based recall, but it necessarily narrows the scope of the claims that can be made. The tasks should therefore be interpreted as probes of computational structure rather than as direct measurements of broad reasoning ability. More generally, the decomposition adopted in this work is most naturally aligned with the SoT interpretation of reasoning, in which reasoning depends heavily on the maintenance and manipulation of token-level intermediate states. However, SoT is not the only possible account of reasoning behavior, and more holistic views may involve additional mechanisms or abstractions not captured by the primitives isolated here. Accordingly, the present benchmark should not be interpreted as proposing an exhaustive decomposition of reasoning itself, but rather as isolating a set of primitives that appear especially relevant for the state-based recall setting examined in this work. The present results therefore do not establish that the same trends will necessarily transfer to more naturalistic domains such as long-document question answering, mathematical reasoning, or tool-use settings. In addition, the experiments were conducted under a fixed inference budget. Although most failed Think trajectories appear to enter repetitive “doom loops” rather than making measurable progress toward a solution, it remains possible that different budget regimes could alter some of the observed comparisons. A more systematic analysis of budget scaling would help clarify whether the main differences observed here reflect representational limitations or simply the increasing computational cost of sustaining long reasoning traces. Finally, the paper evaluates token-space reasoning only indirectly. While the overall empirical pattern is broadly con-

¹Although, we were unable to find hybrid and non-hybrid models that would allow for this level of fairness in terms of comparison, i.e., models mainly differ in terms of architecture, but nothing else.

sistent with the SoT interpretation, we do not directly compare natural-language reasoning traces against compressed traces, nor do we evaluate architectures that partially externalize reasoning into latent representations. Such comparisons remain important directions for future work to disentangle the relative contributions of state externalization, architectural design, and inference efficiency to the reasoning gains observed in this work.

6 Ethics Statement

This research was conducted using publicly available models, and did not involve human subjects or personally identifiable information. The computational resources used were provided by the authors’ institution, and the environmental impact of training and inference was minimized to the best of our abilities, while the total cost (i.e., energy consumption and carbon emissions equivalents) has been reported. The findings of this research are intended to advance understanding of reasoning capabilities in language models and do not have direct applications that could be harmful. However, as with all research in artificial intelligence, there is a potential for misuse, and we encourage responsible use of these findings in the development of future AI systems.

Acknowledgments

[redacted for peer review]

References

- Elie Bakouch, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, Lewis Tunstall, Carlos Miguel Patino, Edward Beeching, Aymeric Roucher, Aksel Joonas Reedi, Quentin Gallouédec, and 1 others. 2025. Smollm3: smol, multilingual, long-context reasoner. *Hugging Face Blog*.
- Jessie L Christiansen, Douglas L McElroy, Marcy Harbut, David R Ciardi, Megan Crane, John Good, Kevin K Hardegree-Ullman, Aurora Y Kesseli, Michael B Lund, Meca Lynn, and 1 others. 2025. The nasa exoplanet archive and exoplanet follow-up observing program: Data, tools, and usage. *The Planetary Science Journal*, 6(8):186.
- Tri Dao and Albert Gu. 2024. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*.
- Alexander M Fichtl, Jeremias Bohn, Josefin Kelber, Edoardo Mosca, and Georg Groh. 2025. The end of transformers? on challenging attention and the

rise of sub-quadratic architectures. *arXiv preprint arXiv:2510.05364*.

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. 2025. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv preprint arXiv:2502.05171*.

Subbarao Kambhampati, Kaya Stechly, and Karthik Valmeekam. 2025. (how) do reasoning models reason? *Annals of the New York Academy of Sciences*, 1547(1):33–40.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626.

Alexandre Lacoste, Alexandra Luccioni, Victor Schmidt, and Thomas Dandres. 2019. [Quantifying the carbon emissions of machine learning](#). *Preprint*, arXiv:1910.09700.

Mosh Levy, Zohar Elyoseph, Shauli Ravfogel, and Yoav Goldberg. 2025. State over tokens: Characterizing the role of reasoning tokens. *arXiv preprint arXiv:2512.12777*.

Haoyang Li, Yiming Li, Anxin Tian, Tianhao Tang, Zhanchao Xu, Xuejia Chen, Nicole Hu, Wei Dong, Qing Li, and Lei Chen. 2024. A survey on large language model acceleration based on kv cache management. *arXiv preprint arXiv:2412.19442*.

Hanmeng Liu, Zhizhang Fu, Mengru Ding, Ruoxi Ning, Chaoli Zhang, Xiaozhang Liu, and Yue Zhang. 2025. Logical reasoning in large language models: A survey. *arXiv preprint arXiv:2502.09100*.

William Merrill, Yanhong Li, Tyler Romero, Anej Svete, Caia Costello, Pradeep Dasigi, Dirk Groeneveld, David Heineman, Bailey Kuehl, Nathan Lambert, and 1 others. 2026. Olmo hybrid: From theory to practice and back. *arXiv preprint arXiv:2604.03444*.

Sumeet Ramesh Motwani, Daniel Nichols, Charles London, Peggy Li, Fabio Pizzati, Acer Blake, Hasan Hammoud, Tavish McDonald, Akshat Naik, Alesia Ivanova, and 1 others. 2026. Longcot: Benchmarking long-horizon chain-of-thought reasoning. *arXiv preprint arXiv:2604.14140*.

Team Olmo, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heineman, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers, Hamish Ivison, Jacob Morrison, Jake Poznanski, Kyle Lo, Luca Soldaini, Matt Jordan, Mayee Chen, Michael Noukhovitch, Nathan Lambert, Pete Walsh, and 49 others. 2025. [Olmo 3](#). *Preprint*, arXiv:2512.13961.

Liang Wang, Nan Yang, Shaohan Huang, Li Dong, and Furu Wei. 2025. Thinking augmented pre-training. *arXiv preprint arXiv:2509.20186*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, and 1 others. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. 2024a. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*.

Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. 2024b. Parallelizing linear transformers with the delta rule over sequence length. *Advances in neural information processing systems*, 37:115491–115522.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822.

Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, and 1 others. 2026. Glm-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*.

A Scientific Artifacts

This study releases the source code and data needed to reproduce the results. The codebase is available at [redacted for peer review].

B Formal Definitions of Reasoning Primitives

This appendix provides formal definitions of the two reasoning primitives introduced in Section 2.

State-tracking. State-tracking is the ability to maintain and update structured variables over time under sequential transformations. The critical requirement is not merely the storage of initial values, but the preservation of the evolving relational configuration among variables as successive updates are applied. Formally, letting S_t denote the state at step t and U_t the update function applied at that step, the model must compute:

$$S_t = U_t(S_{t-1}) \quad (1)$$

and ensure that S_t accurately reflects the cumulative effect of all updates up to time t . Recurrent and state-space models are well-matched to this regime because they maintain a compact hidden state that is updated incrementally at each step. Standard attention mechanisms are often less naturally aligned with problems whose difficulty lies in preserving the result of many compositional updates, especially when those updates are long-range, indirect, or involve repeated reassignment.

Recall. Recall is the ability to retrieve a specific piece of information from a large context with high precision, often when the relevant signal is sparse relative to the total input. Formally, given a context C of N items and a query q , the model must compute:

$$R = \text{Attention}(q, C) \quad (2)$$

such that R accurately reflects the relevant item in C corresponding to q , especially as N grows large and the relevant signal becomes sparser. This is the regime in which softmax attention is most naturally interpreted as a differentiable associative memory mechanism. Purely recurrent or linear state-space models must compress history into a fixed-dimensional state, making fine-grained retrieval progressively harder over long horizons.

State-based recall. When the two primitives are composed, the retrieval query is not a fixed external input but is itself the output of a state-tracking process. The model must first compute the final state S_T via Equation 1 and then use that state as the query for retrieval via Equation 2:

$$R = \text{Attention}(S_T, C) \quad (3)$$

This composition makes the task irreducible to either primitive in isolation: an error in state propagation corrupts the retrieval address, and a failure in retrieval produces the wrong answer even given a correct address. Hybrid architectures have been hypothesized to be more expressive due to their inherent architectural design (Merrill et al., 2026), which combines the inductive biases.

C Additional Results

Figures 5–10 show accuracy and parse rate results for the three task families not shown in the main body: State-based Astro Recall, Dyck Language, and OLMo Original.

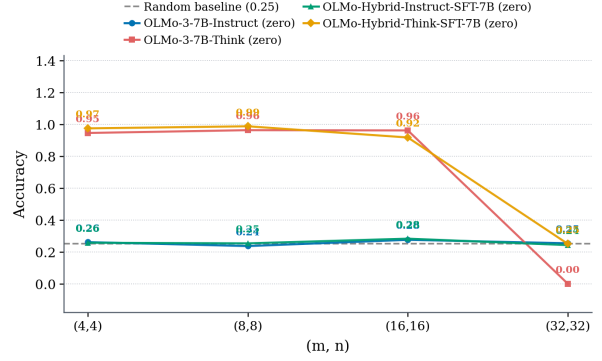


Figure 5: Astro Recall accuracy.

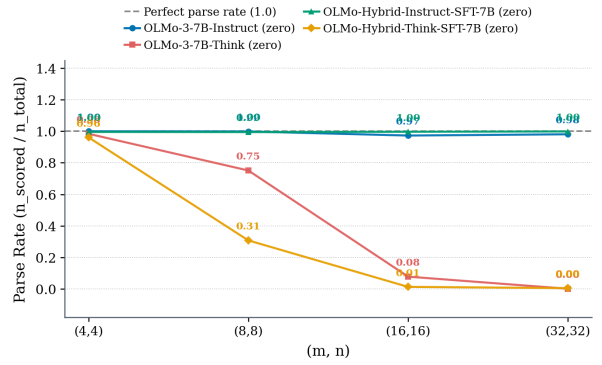


Figure 6: Astro Recall parse rate.

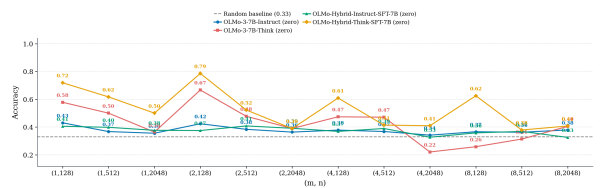


Figure 7: Dyck accuracy.

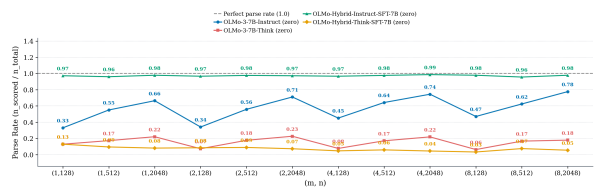


Figure 8: Dyck parse rate.

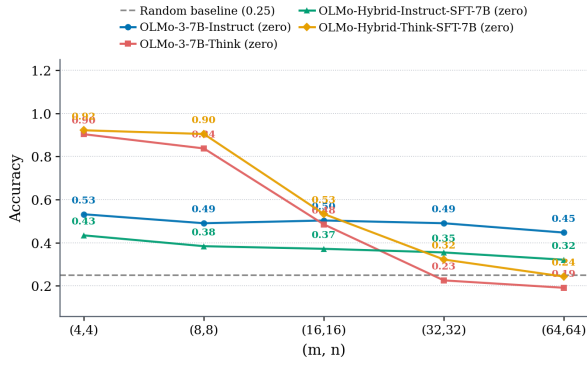


Figure 9: OLMo Original accuracy.

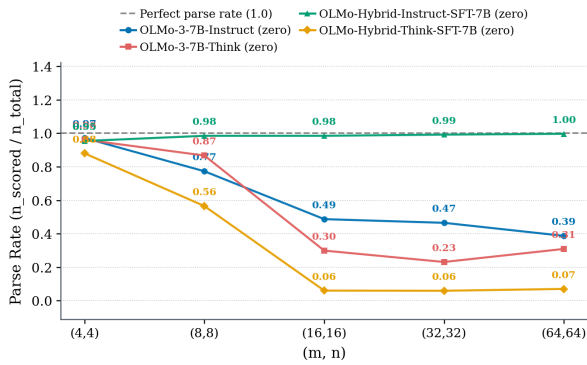


Figure 10: OLMo Original parse rate.

D Task Suite Details and Prompt Examples

This appendix groups the task descriptions and prompt templates for the five task families discussed in the main text.

D.1 Reasoning Primitive Examples

Before turning to the task families themselves, we first give three short canonical examples illustrating the primitives discussed in Section 2: state-tracking, recall, and their composition into state-based recall, as proposed by Merrill et al. (2026).

Example 1. State-Tracking A canonical state-tracking task requires propagating variable bindings across a chain of dependent updates. The variables are initialized with binary values, then updated through a sequence of n dependent swap operations; the final assertion depends on the entire chain of updates. The challenge lies not in retrieving a fixed fact, but in preserving the result of many compositional updates—precisely the regime where recurrent architectures that maintain an incrementally updated hidden state are well matched:

Toxicity Evaluation Prompt

```
a, b, c, d, e = [0, 1, 0, 0, 1]
a, c = c, e
...
assert a == _
```

Example 2. Recall A canonical recall task requires retrieving a specific item from a large array-like context given a precise index. The variable a holds an index into a long list of bits; the assertion depends solely on retrieving the bit at that position. The challenge is to localize the relevant item without losing fidelity as context length grows—the regime where softmax attention functions as a differentiable associative memory mechanism, and where purely recurrent models that compress history into a fixed-dimensional state face increasing difficulty:

Toxicity Evaluation Prompt

```
bits = [0, 1, 0, 0, ...]
a = 42
assert bits[a] == _
```

Example 3. State-Based Recall A canonical state-based recall task combines both primitives. A long list of bits serves as the memory, and the variables a – e are initialized as indices into that list. After n dependent swap operations, the final assertion depends on both correctly tracking the transformed index (state-tracking) and retrieving the bit at that position (recall). The retrieval step is conditioned on the outcome of the state-tracking step, making the task irreducible to either primitive in isolation:

Toxicity Evaluation Prompt

```
bits = [0, 1, 0, 0, ...]
a, b, c, d, e = 36, 23, 12, 2, 56
a, c = c, e
...
assert bits[a] == _
```

D.2 OLMo Original (*olmo_original*)

A synthetic state-based recall task imported directly from (Merrill et al., 2026). A bit array of length m is generated, and m pointer variables are each initialized to a distinct random index. After n simultaneous swap operations, the model must evaluate $\text{bits}[a]$ —combining pointer tracking with binary value lookup. This is the most minimal state-based recall formulation in the suite and serves as the replication baseline.

System Prompt

You are a strict code-execution engine.

Task:

1. You are given a bit array and pointer variables (a, b, c, ...).
2. You are given a sequence of simultaneous swap assignments.
3. You must track the pointer values through every swap and then look up the correct bit in the array.

Rules:

- Each swap line uses Python simultaneous assignment: `x, y = y, x`.
- Apply every swap in order; do **not** skip any.
- After all swaps, evaluate `bits[queried variable]`.

Output requirements:

- Return EXACTLY one JSON object, no other text.
- Example Output: `{"answer": "A | B | C | D"}`

Example Prompt

- `bits = [0, 1, 1, 0]` (4 bits)
- `a, b, c, d = 2, 0, 3, 1` (indices 0..3)
- Swaps:
 - `a, b = b, a`
 - `b, c = c, b`
- Then evaluate: `bits[a]` (expected 0 or 1)

Options

- A) 1
- B) 0
- C) 2
- D) 3

D.3 Dyck Language (*dyck*)

The model is given a bracket sequence drawn from the pairs `()`, `[]`, and `{}`, with one closing token masked, and must identify the correct closing token by maintaining a stack. Difficulty is controlled by m (stack depth at the query position) and n (total sequence length). This is the only task in the suite containing no retrieval component—it stresses pure state-tracking in isolation.

System Prompt

You are a strict language validator for Dyck expressions.

Rules:

- A Dyck expression uses bracket pairs: `()`, `[]`, `{}`.
- Every opening bracket must be closed by its exact matching closer.
- Brackets must be closed in the correct order (last opened = first closed).

Task:

1. You are given a Dyck expression with one token masked as `_`.
2. You must determine which token `_` must be to keep the expression valid.

How to solve:

- Read the sequence token by token from left to right.
- Maintain a stack: push every opener `(`, `[`, `{` onto the stack.
- When you see a closer `)`, `]`, `}`, pop the top of the stack and require a matching pair.
- The masked token `_` must match the opener currently on top of the stack at that position.
- Do NOT skip any token. Do NOT guess based on surrounding tokens alone.

Output requirements:

- Return EXACTLY one JSON object, no other text.
- Example Output: `{"answer": "A | B | C"}`

Example Prompt

Expression: `( [ { _ ] ) ( )`
Bracket pairs: `()` `[]` `{}`.
Every opener must be closed by its matching closer in the correct order.
What token must replace `_` at the indicated position?

Options

- A) `)`
- B) `}`
- C) `]`

D.4 State-based Astro Recall (*astro*)

Each instance presents the model with a Markdown table of m real exoplanet rows. The data is taken from NASA Exoplanet Archive ([Christiansen et al.](#),

2025). Named variables are initialized to the orbital period values of those rows, then subjected to n simultaneous swap operations. The model must determine which planet name corresponds to the queried variable after all swaps. This couples state-tracking (pointer propagation through swaps) with recall (lookup in a semantically structured multi-attribute table) and tests whether the state-based recall challenge persists when embedded in a more naturalistic retrieval context.

System Prompt

You are a precise reasoning assistant.
You will be given:

1. A table of exoplanet data with planet names and various properties.
2. Variable assignments mapping variable names to values from a specific column.
3. One or more swap operations (Python-style simultaneous assignment).
4. A question asking which planet corresponds to a variable after all swaps.

How to solve:

- Each variable ((a,b,c,...)) starts assigned to a specific value from the table and therefore corresponds to a specific planet.
- Each swap line exchanges the values of two variables simultaneously: (x, y = y, x) means (x) gets (y)'s current value and (y) gets (x)'s current value.
- Track which value each variable holds after every swap.
- At the end, find which planet in the table has the value that the queried variable currently holds.
- Apply every swap in order. Do NOT skip any.

Output requirements:

- Return EXACTLY one JSON object.
- No extra text.

Format: {"answer": "A | B | C | D"}

Example Prompt

Planet	Orbital Period (days)
Kepler-1	2.684
Kepler-2	8.798
Kepler-3	5.508
Kepler-4	2.644

Consider the following Orbital Period (days):

a, b, c, d = 2.684, 8.798, 5.508, 2.644

Consider the following swapping:

- a, b = b, a
- b, c = c, b

The Planet with the Orbital Period (days) = a is:

Options

- A) Kepler-2
- B) Kepler-1
- C) Kepler-3
- D) Kepler-4

D.5 Collision Simulator (collisions)

Each instance places m particles in a one-dimensional system with distinct initial velocities drawn from a large integer pool. A sequence of n pairwise elastic collisions is applied in order; equal-mass particles exchange velocities on contact. Unlike the simultaneous swaps in the astro task, each collision here conditions all subsequent ones, creating a chain of compounding sequential dependencies. This makes the task more adversarial to state maintenance: a single error at step k corrupts all later steps.

System Prompt

You are a strict state-tracking engine for collision systems.

Task:

1. You are given particles with initial velocities.
2. You are given a sequence of pairwise collisions.

Core rule (MUST be applied exactly):

- When two equal-mass particles collide, they EXCHANGE velocities.
- This is equivalent to swapping their velocity values simultaneously.
- If (A) collides with (B): (new_A = old_B, new_B = old_A). Both update at once.

Reasoning requirements:

- Maintain an explicit mapping: particle ($\rightarrow$) current velocity.
- Apply each collision in order.
- After each collision, update **both** particles' velocities before moving to the next.
- Do NOT skip steps.

659

- Do **NOT** infer physics beyond the given rule.

Output requirements:

- Return **EXACTLY** one JSON object.
- No extra text.
- Example Output: `{"answer": "A | B | C | D"}`

Example Prompt

Consider a one-dimensional system where all particles move along a line.

Key rule:

- When two equal-mass particles collide elastically, they exchange velocities.

Initial velocities

- (A = 142)
- (B = 57)
- (C = 831)

Collisions

1. (A) collides with (B)
2. (B) collides with (C)

Question

What is the velocity of particle (A) after all collisions?

Options

- A) 142
- B) 57
- C) 831
- D) 204

660

661

D.6 DAG Arithmetic (*dag_arithmetic*)

Following (Motwani et al., 2026), m input variables are assigned integer values and n computation layers introduce new variables via addition or subtraction, with later layers able to reference any prior variable. The flat, multi-hop retrieval structure over a growing variable registry places different demands on the model than the sequentially chained updates of the collision task.

System Prompt

You are a strict arithmetic computation engine.

Task:

1. You are given a set of input variables with integer values.
2. You are given a sequence of computation steps

670

organized in layers.

3. Each step computes a new variable from one or two previous variables using addition or subtraction only.
4. You must trace every computation in order and track all variable values.

Rules:

- Apply every step in order. Do not skip any.
- Use integer arithmetic throughout; all values are integers.
- Each step may reference any previously computed variable, not only variables from the immediately preceding layer.
- Keep a running record of *all* variable values at all times.
- After all steps, report the value of the queried variable.

Output requirements:

- Return **EXACTLY** one JSON object, no other text.
- Example Output: `{"answer": "A | B | C | D"}`

671

Example Prompt

DAG arithmetic computation

Input variables:

- $a = 3$
- $b = 7$

Computation steps:

Layer 1:

- Step 1: $v1_0 = a + b$
- Step 2: $v1_1 = b - 2$

Layer 2:

- Step 3: $v2_0 = v1_0 + v1_1$
- Step 4: $v2_1 = a + v1_0$

What is the value of $v2_1$ after all computations?

Options

- A) 10
- B) 13
- C) 15
- D) 7

672

E Infrastructure and Evaluation Details

This appendix documents our compute infrastructure, software stack, and inference configuration.

E.1 HPC Cluster

All inference runs were carried out on the [redacted for peer review]. All experiments were conducted using a single node equipped with $8 \times$ NVIDIA A40 GPUs (48 GB HBM2 each), with intra-node GPU communication via NVLink and inter-node communication via Mellanox InfiniBand NDR at 200 Gb/s.

E.2 Software Stack

Our codebase uses vLLM (Kwon et al., 2023) for high-throughput inference and LLM-based evaluation. All experiments were executed in an environment based on CUDA/12.6.0 and Python/3.12.3. The experimental pipeline, including task generation, inference orchestration, parsing, and evaluation utilities, was implemented entirely within this software stack to ensure reproducibility and consistency across runs. Source code implementation details, together with links to the released scientific artifacts, are provided in Appendix A.

E.3 Inference Configuration

All models were served using vLLM (Kwon et al., 2023) with a batch size of 8 and a maximum model context length of 28,000 tokens. Generation hyperparameters varied by model variant as shown in Table 2.

Parameter	Think variants	Instruct variants
Temperature	default	default
Max model len	28000	28000
Max tokens	4000	256
Batch size	8	8

Table 2: Inference generation parameters for Think and Instruct model variants.

E.4 Resource Consumption

We report a conservative GPU-only estimate of the computational footprint of our experiments. All inference runs were executed on a single node with $8 \times$ NVIDIA A40 GPUs, each with a maximum power draw of 300 W, over a total runtime of 7 days (168 hours). Under this simplifying assumption, the total GPU power draw is 2.4 kW, yielding an estimated energy consumption of 403.2 kWh. Using

Property	Value
Parameters	7B
Training data	Dolma 3
Post-training data	Dolci
Training tokens	6T
Post-training stages	SFT $\rightarrow$ DPO $\rightarrow$ RLVR

Table 3: Architectural and training properties shared across all four models.

Property	OLMo3	OLMo3 Hybrid
Architecture type	Full transformer	Hybrid (attention + recurrent)
Recurrent layer	None	Gated DeltaNet (GDN)
Layer pattern	$32 \times$ attention	$3 \times$ GDN + $1 \times$ attention, repeated 25%
Attention fraction	100%	
Hidden size (d_{model})	4096	3840
Attention heads	32	30
Head dimension	128	256 (doubled for DeltaNet)
Positional encoding	RoPE	RoPE disabled

Table 4: Key architectural differences between the OLMo3 and OLMo3 Hybrid model families. Both families are matched on training data, optimization schedule, and post-training recipe.

an average carbon intensity of 0.37 kgCO₂e/kWh for the German electricity grid, this corresponds to approximately 149.2 kgCO₂e. This should be interpreted as a lower-bound estimate in the spirit of a simplified CodeCarbon-style accounting, since it excludes additional contributions from CPUs, RAM, storage, cooling, and other system-level overheads. These calculations are based on the work of Lacoste et al. (2019).

F Model Architecture Details

Table 3 summarizes the architectural properties shared across all four models evaluated in this study, and Table 4 highlights the key differences between the transformer and hybrid model families.